



Lab Assessment Report

Only for the course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	25
Level of Content						
Analysis						
Development						
Accuracy						
Total obtained mark						
Comments						

Semester: Fall 2025

Student Name: Afzal Hossain Raju

Group: 40-D

Student ID:

Batch: 40

Section: 40-D

Course Code: SE 334

Course Name: Artificial Intelligence Lab

Course Teacher Name: Ms. Nusrat Jahan

Designation: Lecturer

Submission Date: 16-12-2025

Lab Report

1. Structured Programming in Python

1.1 Sequence

```
a = 10
b = 20
sum = a + b
print(sum)
```

30

1.2 Selection (Decision Making)

```
marks = 75

if marks >= 80:
    print("A+")
elif marks >= 60:
    print("A")
else:
    print("Fail")
```

A

1.3 Iteration (Looping)

```
for i in range(5):
    print(i)
```

0
1
2
3
4

```
i = 0
while i < 5:
    print(i)
    i += 1
```

0
1
2
3
4

1. OOP Principles in Python

2.1 Class and Object

```
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Afzal")
print(s1.name)

Afzal
```

2.2 Encapsulation

```
class Account:
    def __init__(self):
        self.__balance = 1000

    def get_balance(self):
        return self.__balance
```

2.3 Abstraction

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

2.4 Inheritance

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

2.5 Polymorphism

```
class Cat:
    def sound(self):
        print("Meow")

class Dog:
    def sound(self):
        print("Bark")
```

```
for animal in (Cat(), Dog()):  
    animal.sound()
```

Meow
Bark

1. Informed Search

A* Algorithm (Weighted Graph)

```
import heapq  
  
def astar(graph, start, goal, heuristic):  
    pq = [(0, start)]  
    visited = set()  
  
    while pq:  
        cost, node = heapq.heappop(pq)  
        if node == goal:  
            return cost  
        if node not in visited:  
            visited.add(node)  
            for neighbor, weight in graph[node]:  
                heapq.heappush(pq, (cost + weight +  
heuristic[neighbor], neighbor))
```

1. Uninformed Search

BFS vs DFS (Unweighted Graph) BFS (Queue – Level Order)

```
from collections import deque  
  
def bfs(graph, start):  
    visited = set()  
    q = deque([start])  
  
    while q:  
        node = q.popleft()  
        if node not in visited:  
            print(node)  
            visited.add(node)  
            q.extend(graph[node])
```

DFS (Stack – Depth First)

```
def dfs(graph, start, visited=set()):  
    if start not in visited:  
        print(start)  
        visited.add(start)
```

```
for n in graph[start]:  
    dfs(graph, n, visited)
```

1. Machine Learning Models

5.1 Linear Regression (Continuous Output)

```
from sklearn.datasets import fetch_california_housing  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error  
  
data = fetch_california_housing()  
X = data.data  
y = data.target  
  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2)  
  
model = LinearRegression()  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)  
  
mse = mean_squared_error(y_test, y_pred)  
print("Mean Squared Error:", mse)  
from sklearn.datasets import fetch_california_housing  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error  
  
data = fetch_california_housing()  
X = data.data  
y = data.target  
  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2)  
  
model = LinearRegression()  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

Mean Squared Error: 0.5636651478154122

Mean Squared Error: 0.5055960707694028

5.2 Logistic Regression (Binary Classification)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

Accuracy: 0.9473684210526315
```

5.3 Random Forest

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
```

```
test_size=0.2)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.9649122807017544
```

5.4 Support Vector Machine (SVM)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

model = SVC(kernel='linear')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.956140350877193
```

5.5 K-Means Clustering

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

data = load_iris()
X = data.data
```

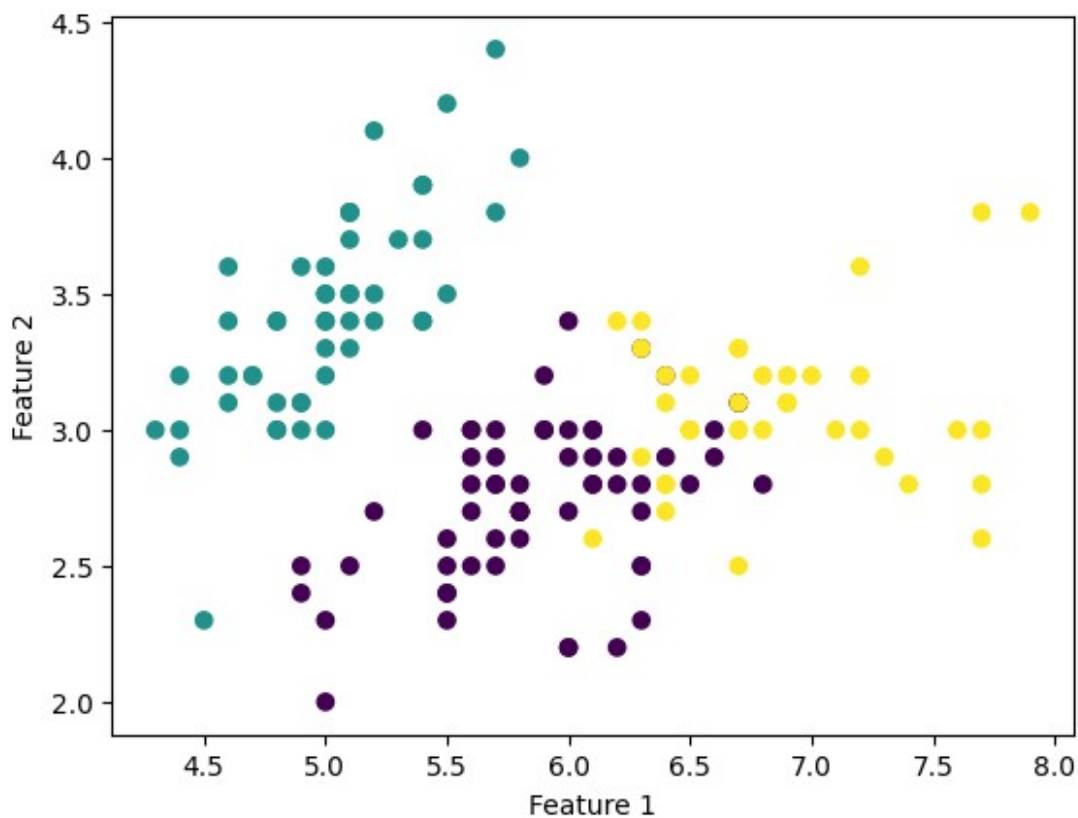
```

kmeans = KMeans(n_clusters=3)
kmeans.fit(X)

labels = kmeans.labels_

plt.scatter(X[:,0], X[:,1], c=labels)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```



5.6 Decision Tree (with Visualization)

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

data = load_iris()
X = data.data
y = data.target

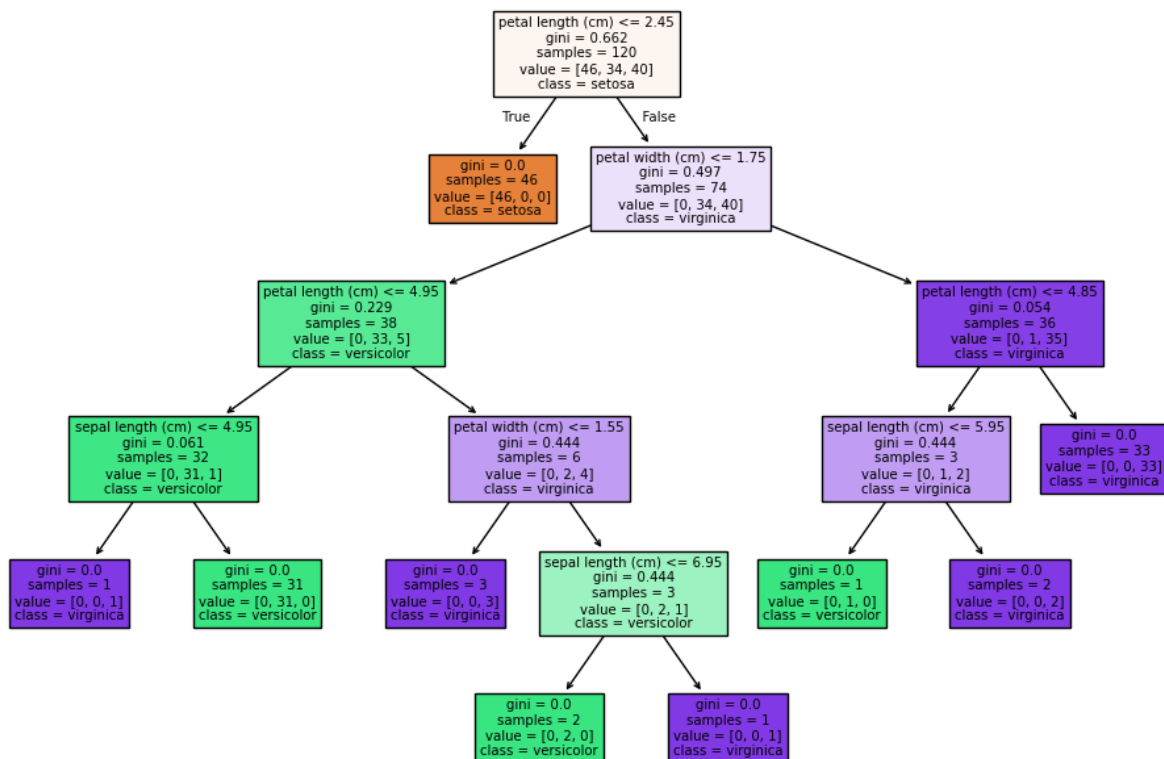
```



```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
```

```
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

```
plt.figure(figsize=(12,8))
plot_tree(model, feature_names=data.feature_names,
          class_names=data.target_names,
          filled=True)
plt.show()
```



5.7 ML Project

California House Price Prediction

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2)
```

```
)

model = LinearRegression()
model.fit(X_train, y_train)

LinearRegression()
```

1. Deep Learning Models

6.1 ANN – Counterfeit Bill Prediction

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(64, activation='relu', input_shape=(8,)))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=10)

Epoch 1/10

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

516/516 ————— 2s 2ms/step - loss: 9829.1270
Epoch 2/10
516/516 ————— 1s 2ms/step - loss: 8.5655
Epoch 3/10
516/516 ————— 1s 2ms/step - loss: 2.4513
Epoch 4/10
516/516 ————— 1s 2ms/step - loss: 1.2086
Epoch 5/10
516/516 ————— 2s 3ms/step - loss: 0.9415
Epoch 6/10
516/516 ————— 2s 2ms/step - loss: 0.9296
Epoch 7/10
516/516 ————— 1s 2ms/step - loss: 0.7987
Epoch 8/10
516/516 ————— 1s 2ms/step - loss: 0.7038
Epoch 9/10
516/516 ————— 1s 2ms/step - loss: 1.1280
Epoch 10/10
516/516 ————— 1s 2ms/step - loss: 0.8944

<keras.src.callbacks.history.History at 0x7f2ec8dcf080>
```

6.2 CNN for MNIST Dataset

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.reshape(-1,28,28,1)/255.0
y_train = to_categorical(y_train)

Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 _____ 0s 0us/step
```

6.3 CNN Model for Image Classification

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense

model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```